

Timing Constraints

This document is heavily based on the excellent book “Constraining Designs for Synthesis and Timing Analysis: A Practical Guide to Synopsys Design Constraints (SDC)” by Gangadharan and Churiwala (Springer, 2013), as well as documentation from Synopsys and Cadence.

1 Introduction

Logic synthesis is the method we use to take a digital design, described in a hardware description language, and efficiently map it to logic gates in a standard cell library. However, the word *efficient* is not as simple as it seems, since efficiency depends on our metrics. In most cases, we have a target frequency that we’d like our design to meet, and that is our primary metric; however, area and power are important secondary targets. Therefore, we must define a set of *constraints* that will be used by the synthesizer to bound the optimization space that it is targeting. The most common syntax for defining constraints is the **Synopsys Design Constraints (SDC)** set of commands, which are a superset of TCL.

2 TCL Extensions for SDC

2.1 Commands for Design Query

Before starting with actual SDC commands for defining constraints, there are several important design query commands that EDA tools utilize to query the design database and provide a collection of entities (cells, ports, pins, nets, clocks). Here is a list of the design query commands:

<code>get_cells</code>	Returns the instance of the design or library cell
<code>get_ports</code>	Returns the input, inout, and output ports of the design
<code>get_pins</code>	Returns the instance of a port of design or library cell pin
<code>get_nets</code>	Returns net connected to the port or pin
<code>get_clocks</code>	Returns clocks in the design
<code>all_inputs</code>	Returns all input and inout (which are theoretically inputs also) ports in the design
<code>all_outputs</code>	Returns all output and inout (which are theoretically outputs also) ports in the design
<code>all_registers</code>	Returns all registers in the design
<code>all_clocks</code>	Returns all clocks in the design
<code>get_libs</code>	Returns the list of libraries
<code>get_lib_cells</code>	Returns the list of cells in the library
<code>get_lib_pins</code>	Returns the list of pins on the library cell
<code>current_design</code>	Sets the scope of the design for the subsequent commands and queries Report the current scope, if no argument is given

2.2 Collections

The value returned from the design query commands are not TCL lists, but rather *collections*. Remember that all variables in TCL are just strings, and so when we want to reference something with meaning, like a port or a net or a cell, we need a more complex data structure, and that is why the collection was created.

2.2.1 Finding out what the collection refers to

The most important command you will need when dealing with collections is something that will turn the database pointer into a string with meaning. For this, we will need to use the `get_object_name` command. The following example will print out the port names line by line:

```
foreach_in_collection port [get_ports *] {
    puts [get_object_name $port]
}
```

2.2.2 Commands for working with collections

The following is a list of commands for working with collections

<code>foreach_in_collection</code>	Iterate over a collection
<code>add_to_collection</code>	Add a list of elements to a base collection. The -unique option removes duplicates from the collection.
<code>append_to_collection</code>	Similar to <code>add_to_collection</code> .
<code>remove_from_collection</code>	Removes a list of elements from an existing collection
<code>compare_collections</code>	Verifies that two collections contain the same objects
<code>copy_collection</code>	Copies a collection
<code>index_collection</code>	Returns a single object from a collection at index <i>N</i> .
<code>sizeof_collection</code>	Returns the number of objects in a collection
<code>filter_collection</code>	Filters objects that meet a certain criterion from the collection
<code>sort_collection</code>	Sorts the collection

2.2.3 Examples of working with collections

- Removing from collection

```
%> set dports [remove_from_collection [all_inputs] CLK]
{"in1", "in2", "in3"}
```

- Filtering collections

```
%> filter_collection [get_cells *] "is_hierarchical == true"
{"i1", "i2"}
%> get_cells * -filter "is_hierarchical == true"
{"i1", "i2"}
```

- Sorting collections

```
%> sort_collection [get_ports *] {direction full_name}
{"in1", "in2", "out1", "out2"}
```

2.3 Common UI Database Entities – An alternative to Collections

The Stylus Common UI database access, provided in newer versions of Cadence tools, provides an alternative method for accessing design entities and their attributes. While you can still use collections (and even interchange between collections and Common UI commands), many advanced features have been implemented in the Stylus approach and they work across all Cadence tools.

Note that one of the big advantages of the new Stylus Common UI usage over traditional SDC commands is that they return *strings* rather than *collections*, which make them much nicer to deal with inside scripts.

As an example of Stylus Common UI usage, as opposed to SDC commands, you could print out all the ports of the design, as follows:

```
foreach port [get_db_ports:*] {
    puts $port
}
```

3 Clock Constraints

3.1 Defining Clocks

3.1.1 Specifying a clock

Clocks are specified with the `create_clock` command. `create_clock` is generally specified on design objects which are used as sources of clock. These source objects can be port, pin, or net.

```
create_clock -period period_value [source_objects] [-name clock_name]
            [-waveform edge_list] [-add] [-comment comment_string]
```

For example:

```
create_clock -period 10 -name clk [get_ports CLK]
```

3.1.2 Virtual Clocks

Virtual clocks are clocks that don't physically exist in the specific block but represent an external trigger that impacts the timing of the block. A virtual clock has no source specified. In reality, it might have a source, but that source could be outside the block being constrained.

```
create_clock -period 10 -name v_clk -waveform {0 5}
```

3.2 Generated Clocks

Generated clocks are usually clocks that are created by a clock divider, clock multiplier (PLL), or through a pre-defined clock gate.

3.2.1 Specifying Generated Clocks

The SDC command for specifying derived clocks in a design is `create_generated_clock`.

```
create_generated_clock [source_objects] -source clock_source_pin
                    [-master_clock master_clock_name] [-name generated_clock_name]
                    [-edges edge_list] [-divide_by factor] [-multiply_by factor]
                    [-invert] [-edge_shift shift_list] [-duty_cycle percent]
                    [-combinational] [-add] [-comment comment_string]
```

3.2.2 Defining a clock divider

An example 2-bit counter, used as a clock divider, is shown in Figure 1. To define the clocks, we can use the following SDC commands:

```
create_clock -period 10 -name CLK [get_ports CLK]
create_generated_clock -name LSB -source [get_port CLK] \
                    -divide_by 2 [get_pins FF1/Q]
create_generated_clock -name MSB -source [get_pins FF1/Q] \
                    -divide_by 2 [get_pins FF2/Q]
```

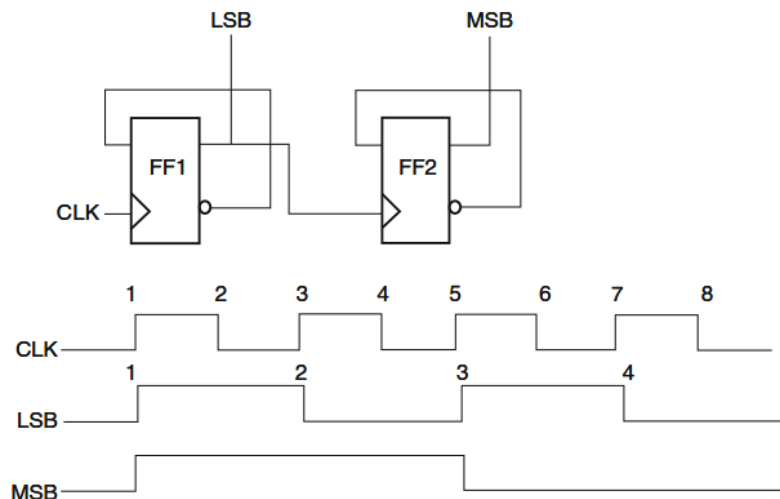


Figure 1: Two-bit counter generating divide-by-two clock divider

3.3 Logically Exclusive Clocks

Often clocks are asynchronous to each other, such as in the example of Figure 2, where either the clock sourced from C1 or from C2 is selected during a certain operating mode, such that no timing path exists between the two clocks. These clocks are considered logically exclusive.

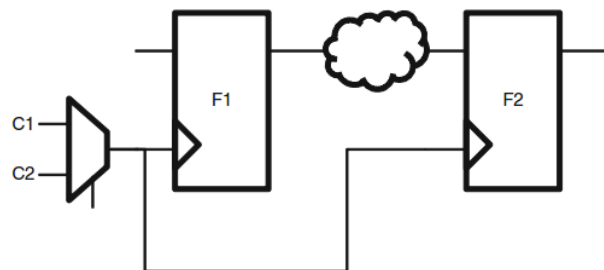


Figure 2: Logically exclusive clocks sourced at different pins of a clock mux.

One way to make sure that timing constraints are not checked between the two clocks is to set a false path between them (see false paths, below). But SDC actually has a better conceptual way to do this, which is to define the clocks as logically exclusive, using the `set_clock_group` constraint:

```
create_clock -period 10 -name C1 -waveform {0 5} [get_ports C1]
create_clock -period 20 -name C2 -waveform {0 12} [get_ports C2]
set_clock_groups -logically_exclusive -group C1 -group C2
```

3.4 Defining Clock Jitter

While clock skew is something that is measured accurately post-CTS, jitter is actually a function of random variables and of the actual clock source (such as the PLL). Therefore, to set margins for

clock jitter, the `set_clock_uncertainty` command is often used to add a window around the actual clock edge and always assume the worst case:

```
set_clock_uncertainty 0.5 [get_clocks C1]
```

or to just define clock uncertainty for setup checks:

```
set_clock_uncertainty -setup 0.5 [get_clocks C1]
```

3.5 Defining an ideal clock or other high fanout network

During synthesis, high-fanout nets, such as the clock network, reset network, or scan mode pins will drive many pins, but without placement information, we do not want to optimize them. This will be done during place and route, but for synthesis, we should define these as ideal networks:

```
set_ideal_network [get_ports CLK]
set_ideal_network [get_ports RST]
set_ideal_network [get_ports SCAN_MODE]
```

3.6 Other clock constraints

3.6.1 Set_clock_transition

Specifies a certain timing transition to be assumed on an otherwise ideal clock. This is unnecessary after clock tree synthesis.

```
set_clock_transition 0.3 [all_clocks]
```

3.6.2 Set_clock_latency

Sets an insertion delay from the clock source until it reaches the clock sinks. This is unnecessary after clock tree synthesis.

```
set_clock_latency -source 0.5 [get_clocks C1]
```

3.6.3 set_timing_derate

```
set_timing_derate [-cell_delay] [-cell_check] [-net_delay]
                  [-data] [-clock] [-early] [-late] [-rise] [-fall]
                  derate_value [object_list]
```

3.6.4 set_wire_load_model

```
set_wire_load_model -name model_name
                   [-library lib_name] [-min] [-max] [object_list]
```

3.6.5 set_propagated_clock

Specifies that delays be propagated through the clock network to determine latency at register clock pins. If not specified, ideal clocking is assumed. In general, the place and route tool will often automatically turn this on after clock tree synthesis.

```
set_propagated_clock [all_clocks]
```

4 Interface Timing Constraints

4.1 Input/Output Delays

Once the clock constraints have been applied, all the register to register paths can be timed. Now the delay constraints have to be applied on non-clock ports. If input and output port constraints are not specified, timing analysis tools assume a highly optimistic timing requirements on the interfaces. They assume the combinational logic inside the block can have the entire period to itself and leave nothing for the portion of the signal outside the block.

In the example of Figure 3, we see two modules, B1 and B2 with a timing path between them, from F1 to F2. Assume they are clocked by the same clock, but are implemented hierarchically, and therefore optimized separately. In order to meet (setup) timing, during the timing optimization of B1, the tool needs to be aware of the path delays of C2 and C3, and therefore, needs to define an *output delay*. Similarly, during the optimization of B2, the tool needs to be aware of the path delays through C1 and C2, and therefore, we need to define an input delay.

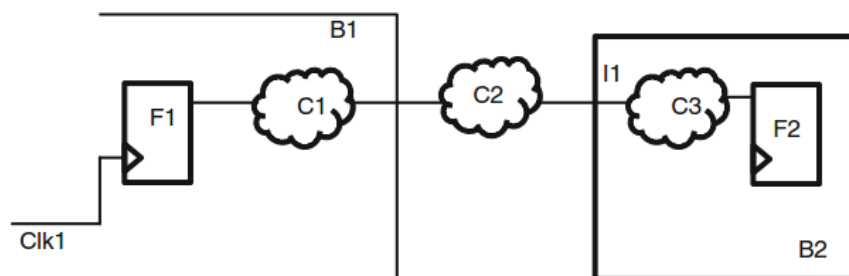


Figure 3: Conceptual hierarchical design with cross-module paths that require interface timing constraints.

4.1.1 set_input_delay

The input delay is the time to assume that it took a signal to arrive at the input due to logic outside the current block:

```
set_input_delay [-clock clock_name] [-clock_fall]
                [-level_sensitive] [-rise] [-fall] [-max] [-min]
                [-add_delay] [-network_latency_included]
                [-source_latency_included]
                delay_value      port_pin_list
```

For example:

```
set_input_delay -clock CLK1 7.0 [get_ports I1]
```

Pay attention that the input delay is set relative to a clock. But there is no actual clock that is toggling the input, and therefore, after CTS, if we propagate the clock, it will add positive skew to the flops that sink inputs (an optimistic situation). Therefore, one approach is to define a virtual clock that toggles the inputs (and outputs) and following CTS, to set an insertion delay on the virtual clock, which is similar to the insertion delay of the clock it is feeding.

4.1.2 set_output_delay

The output delay is the time to assume that it takes a signal to propagate from the output port until reaching the sink register that is outside the current logic block:

```
set_output_delay [-clock clock_name] [-clock_ fall]
                [-level_sensitive] [-rise] [-fall] [-max] [-min]
                [-add_delay] [-network_latency_included]
                [-source_latency_included]
                delay_value      port_pin_list
```

For example:

```
set_output_delay -clock CLK1 7.0 [get_ports OUT1]
```

Note that the same comment as before holds for output delays and the clocking of the port. In this case, however, we would get a pessimistic result, since the skew is negative relative to a zero latency virtual clock.

4.2 Max/Min Delay Constraints

Min and Max delay constraints are not necessarily interface constraints, as they can be used to constrain any path inside the design. However, I decided to describe them here, as they can be used as a clock independent alternative to input/output delays to constrain the interface timing paths.

4.2.1 Command syntax

The syntax for `set_max_delay` and `set_min_delay` is equivalent:

```
set_max_delay [-rise] [-fall] [-from from_list] [-to to_list]
              [-through through_list] [-rise_from rise_from_list]
              [-rise_to rise_to_list] [-rise_through rise_through_list]
              [-fall_from fall_from_list] [-fall_to fall_to_list]
              [-fall_through fall_through_list]
              delay_value [-comment comment_string]
```

4.2.2 Example of min/max delay interface constraining

As mentioned above, min/max delays can be used to constrain interfaces independent of the clock cycle. The recommended option is actually to use the `-max` and `-min` options of `set_input_delay`/`set_output_delay`, but since their usage is less clear, `set_max_delay` is often found. For example, take the circuit of Figure 4. To constrain the input-to-output combinational path of I1 to O1, you could define the following:

```
set_max_delay -from [get_ports I1] -to [get_ports O1] 8.0
```

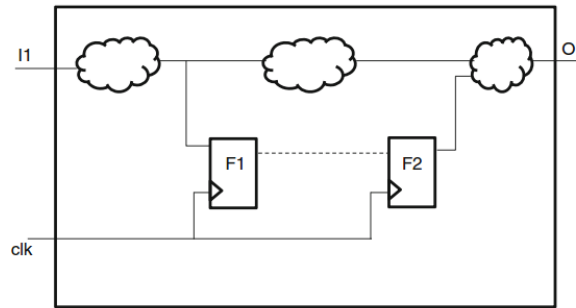



Figure 4: Example of module with input-to-output timing path

4.3 Modeling input/output behavior

4.3.1 set_driving_cell

In order to provide a reasonable transition at the input port, a good way is to assume that it was driven by a certain standard cell from the library. Therefore, if the input has a large wireload, the transition will be slow and the delay of the input port's fanout will be long:

```
set_driving_cell -lib_cell INVX1 [all_inputs]
```

4.3.2 set_input_transition

Alternatively, you can just give a default transition that will be assumed at the input to each port:

```
set_input_transition 0.5 [all_inputs]
```

4.3.3 set_load

While for input ports, we need a transition value to calculate the propagation delay of the next stage, for output ports, we need a load to calculate the delay of the preceding stage. This can be done by providing a value (in capacitance, i.e., pF):

```
set_load 0.3 [all_outputs]
```

Note that just giving a number is not a great methodology. Instead, query the library for the input capacitance of a representative standard cell, and use that as the load value!

4.4 Other (less common) SDC commands for ports

set_drive – sets the resistance of the driver of the input. Not highly recommended.

set_fanout_load – provides a default number for the fanout of an output port to calculate the delay according to the wireload model.

5 Timing Exceptions

5.1 False Paths

5.1.1 False Path Syntax

There are many cases, where timing doesn't need to be checked or it doesn't make sense to check timing. In these cases, the `set_false_path` command is used to state the path or paths that should not be checked or optimized:

```
set_false_path [-setup] [-hold] [-rise] [-fall]
  [-from from_list] [-to to_list] [-through through_list]
  [-rise _from rise_from_list] [-rise_to rise_to_list]
  [-rise_through rise_through_list] [-fall_from fall_from_list]
  [-fall_to fall_to_list] [-fall_through fall_through_list]
  [- comment comment_string]
```

For example, a possible false path would be the falling edge of an asynchronous reset signal:

```
set_false_path -fall_from [get_ports RST_]
```

5.1.2 Types of false paths

A straightforward type of false path is a *combinational false path*, which cannot logically occur due to the functionality of the logic path. For example, in the circuit of Figure 5, the inverted logic relationship between the selectors of mux1 and mux2 make it impossible for a path to go through both muxes A pins or both muxes B pins. It is very computationally intensive to find all of these cases, and so it would be beneficial to mark this as a false path:

```
set_false_path -from [get_ports A] -through [get_pins mux1/A] \
  -through [get_pins mux2/A] -to [get_ports B]
```

or simply

```
set_false_path -through [get_pins mux1/A] -through [get_pins mux2/A]
```

and similarly

```
set_false_path -through [get_pins mux1/B] -through [get_pins mux2/B]
```

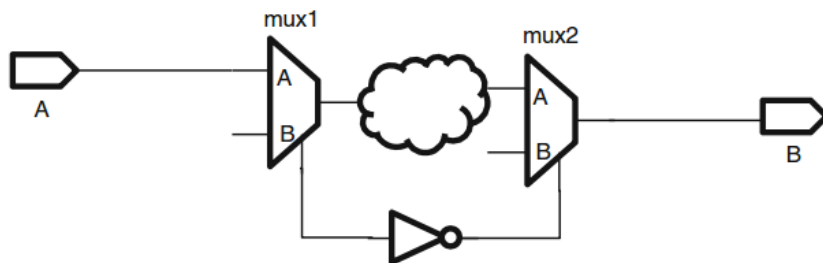


Figure 5: Example of a combinational false path

Other types of false paths are *sequential false paths*, *dynamically sensitized false paths*, timing false paths, false paths due to bus protocol, and false paths between virtual and real clocks. For more information, please turn to Gangadharan and Churiwala.

Be very careful using false paths. To start, if your definition is wrong, you may be making a real timing violation disappear, and you won't even see it during signoff. Second, a false path will not be optimized, and therefore, you may have really terrible timing on it. It's better practice to use a multi-cycle or max-delay, when a timing path is not important but not truly (logically or sequentially) false. For example, a reset network can be defined multi-cycle and the same is true for a path leaving a register that is set upon configuration and held constant throughout operation.

5.1.3 Disabling Timing

In some cases, we don't want to set a false path, but actually disable the timing for a specific timing arc. A good example of this is when we have a combinational feedback loop (such as a ring oscillator). The timing tool will arbitrarily choose where to *break the timing loop* (since it cannot deal with such a thing) and give a warning, but it is much better to explicitly tell it where to break the loop.

```
set_disable_timing [-frompin_name] [-topin_name]
design_objects
```

5.2 Multi-Cycle Paths

5.2.1 Command syntax

For various reasons, we may want to tell the tool that a certain timing path can take multiple cycles to finish, rather than the default single cycle. For this, we use the [set_multicycle_path](#) command:

```
set_multicycle_path [-setup] [-hold] [-rise] [-fall] [-start] [-end]
  [-from from_list] [-to to_list] [-through through_list]
  [-rise_from rise_from_list] [-rise_to rise_to_list]
  [-rise_through rise_through_list] [-fall_from fall_from_list]
  [-fall_to fall_to_list] [-fall_through fall_through_list]
  path_multiplier [-comment comment_string]
```

For example, to define that a certain path has two cycles to finish:

```
set_multicycle_path -setup \
  -from [get_pins FF1/CP] -to [get_pins FF2/D] 2
```

The default setup timing relationship requires the data to be stable at the sink register at the next rising edge, as illustrated in Figure 6. By applying the command in the example, the capture edge is shifted one cycle to the right, as illustrated in Figure 7.

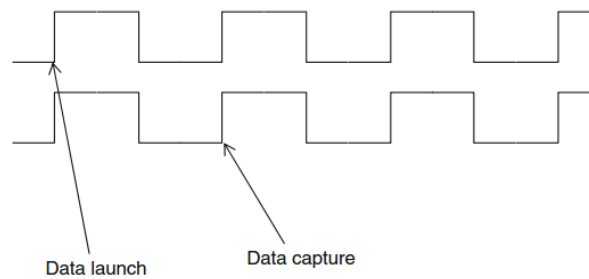


Figure 6: Default setup timing relationship

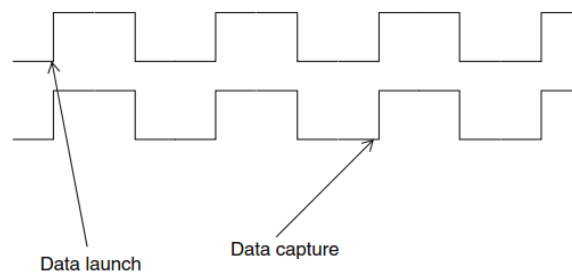


Figure 7: Multi-cycle of 2

5.2.2 Multi-cycles for hold (min) constraints

Oh, boy. Here's another one of those things you just have to do, but you will never quite understand. If you just set a setup multi-cycle, you will completely screw up the hold checks. So just always set the hold multi-cycle to 1 less than the setup multicycle and all will be okay. For example:

```
Set MC_SHIFT 4
set_multicycle_path -setup -from FF1/CP -to FF2/D $MC_SHIFT
set_multicycle_path -hold -from FF1/CP -to FF2/D [expr $MC_SHIFT-1]
```

BEWARE! Don't forget to do this or you will be sorry.

5.2.3 Multi-cycle usage scenario example

Let us consider the circuit of Figure 8. In this example, the data driven by F1 is only sampled when an enable signal is asserted. However, the enable signal is only generated after N cycles, according to the state machine marked by the cloud in the figure. Therefore, the data from F1 only needs to arrive at F3 after N cycles, and there's no need to expedite this path. Therefore, we will constrain it with:

```
set_multicycle_path -from Cs -through F1/Q -to Cd -setup N -end
set_multicycle_path -from Cs -through F1/Q -to Cd -hold N-1 -end
```

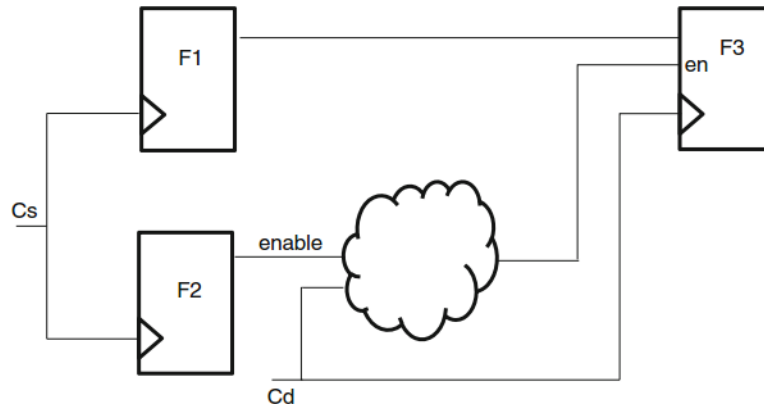


Figure 8: Multi-cycle scenario usage example

5.3 Defining a constant net

A very common situation is that a certain input or register is set to a constant value during some mode of operation. If, for example, an input is set to '0' and drives an AND gate or an enable of a flip flop, it is clear that a false path occurs on one or more timing paths. Since in this case, recognition of false paths is computationally simple, timing tools will perform *constant propagation* and simplify the timing analysis and optimization. Definition of a constant is given in with the following syntax:

```
set_case_analysis value port_or_pin_list
```

A common use case is to load different SDC files for different operating modes, such as functional mode vs. scan mode. In functional mode, there is no need to optimize scan paths and therefore, if the scan enable input is called SE, the following case analysis would be set:

```
set_case_analysis 0 [get_ports SE]
```

6 Constraints for design rules

A final category of SDC commands is the DRC or DRV constraints that can override the tool defaults or the definition in the library (.lib) file. These are taken into consideration during optimization and violation reporting and include the following commands:

- `set_max_capacitance`
- `set_min_capacitance`
- `set_max_transition`
- `set_max_fanout`

This document was provided to you courtesy of Prof. Adam Teman of the EnICS Labs Institute in the Alexander Kofkin Faculty of Engineering at Bar-Ilan University. The content was entirely written and compiled by Prof. Teman (without the help of AI tools), based on knowledge and non-proprietary data. You can find all of Prof. Teman's lectures and materials at the EnICS Labs website:

<https://enicslabs.com/education/>

For any comments or enquiries, Prof. Teman can be reached at adam.teman@biu.ac.il